

# C++ 프로그래밍

## (입출력과 파일처리)

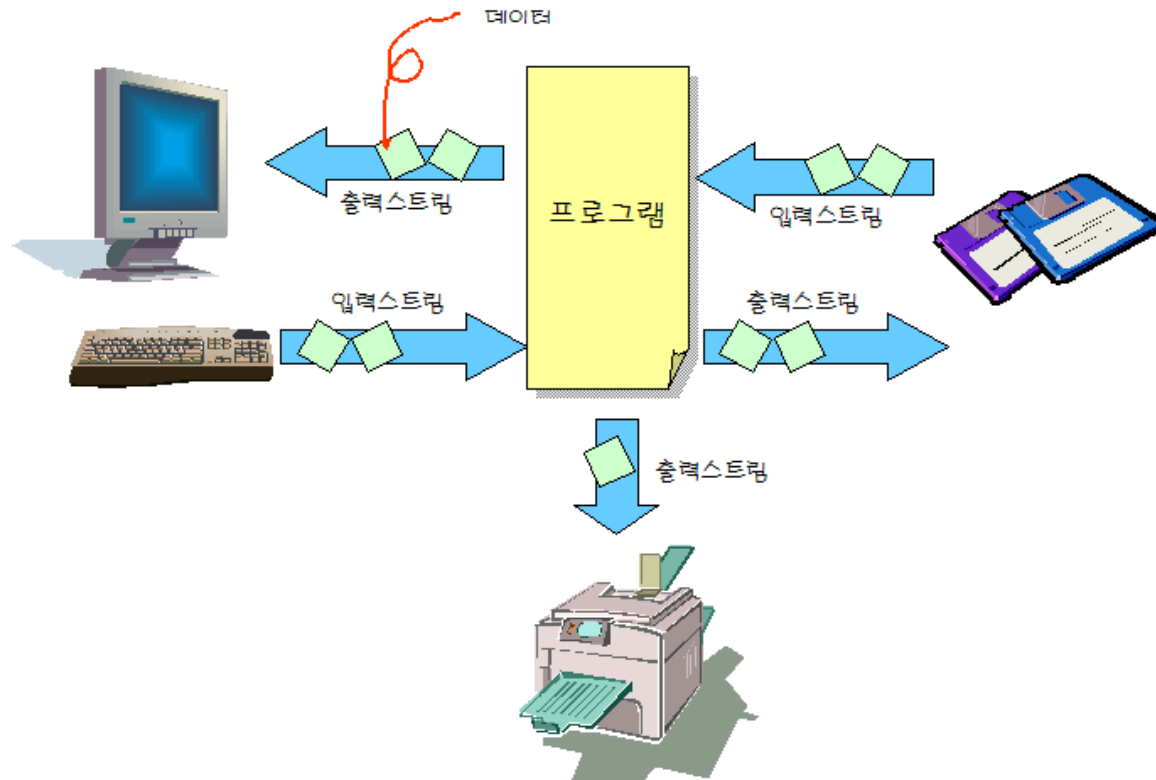
12주차

# 목차

- 파일 입출력
- 텍스트 파일과 이진 파일
- 순차 파일과 랜덤 파일
- 실습
  - 파일 쓰기/읽기
  - ifstream을 이용한 파일 읽기/쓰기
  - 기존파일 뒤에 데이터 추가
  - 이진 데이터 쓰기/읽기

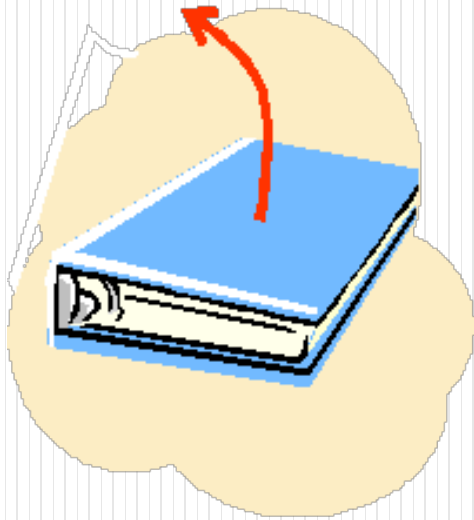
# 파일 입출력

- 스트림(stream)
  - 스트림(stream)은 "순서가 있는 데이터의 연속적인 흐름"이다.
  - 스트림은 입출력을 물의 흐름처럼 간주하는 것이다.

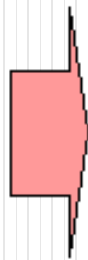


# 파일 입출력 (계속)

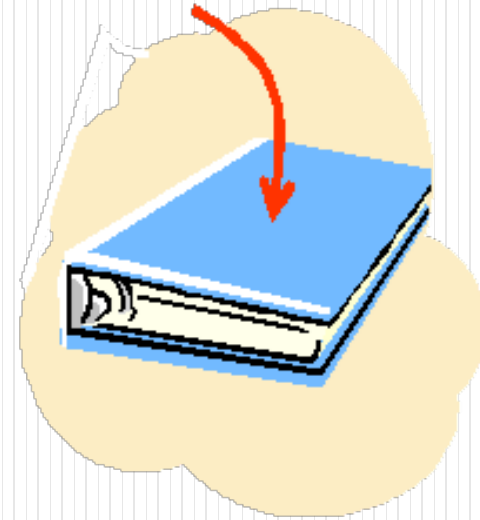
- 파일 처리의 순서
  - 파일을 다룰 때는 반드시 다음과 같은 순서를 지켜야 한다.



파일 열기



파일 읽기와 쓰기



파일 닫기

# 파일 입출력 (계속)

- "<<"과 ">>"을 이용한 입출력
  - 입력

```
ifstream is;  
is.open("test.txt");  
int num;  
is >> num;
```
  - 출력

```
ofstream os;  
os.open("result.txt");  
os << num;
```

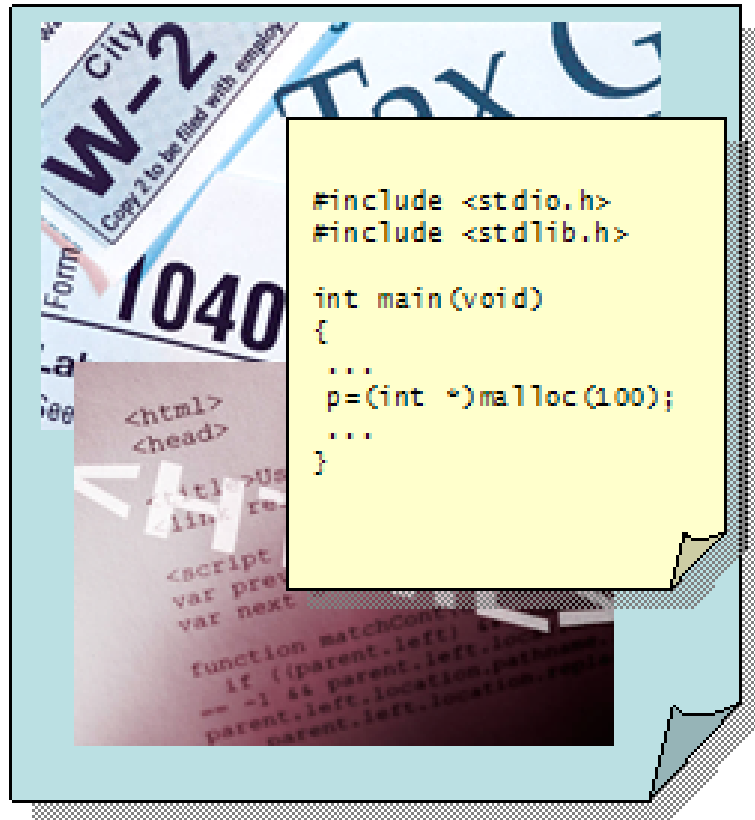
# 파일 입출력 (계속)

- 출력 형식 지성

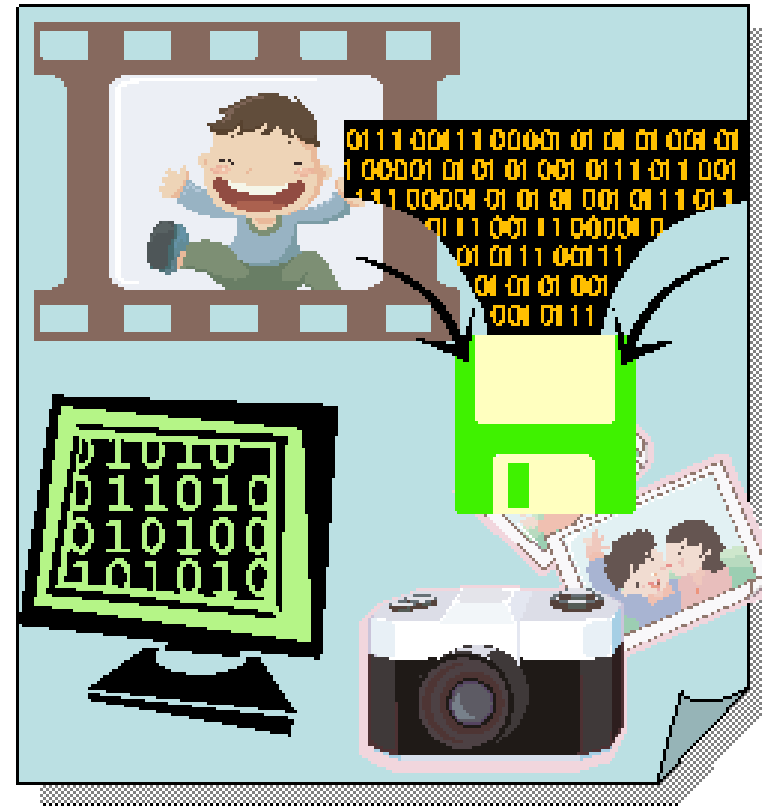
| 플래그                          | 설명                                 |
|------------------------------|------------------------------------|
| <code>ios::fixed</code>      | 고정 소수점 표기법 사용                      |
| <code>ios::scientific</code> | 과학적 표기법 사용(지수를 이용하여 표기)            |
| <code>ios::showpoint</code>  | 소수점을 반드시 표시한다.                     |
| <code>ios::showpos</code>    | 양수 부호를 반드시 출력한다.                   |
| <code>ios::right</code>      | 값을 출력할 때 오른쪽 정렬을 사용한다.             |
| <code>ios::left</code>       | 값을 출력할 때 왼쪽 정렬을 사용한다.              |
| <code>ios::dec</code>        | 값을 출력할 때 10진법을 사용한다.               |
| <code>ios::oct</code>        | 값을 출력할 때 8진법을 사용한다.                |
| <code>ios::hex</code>        | 값을 출력할 때 16진법을 사용한다.               |
| <code>ios::uppercase</code>  | 지수나 16진법으로 표시할 때 대문자를 사용한다.        |
| <code>ios::show</code>       | 8진수이면 앞에 0을 붙이고 16진수이면 앞에 0x를 붙인다. |

```
cout.precision(3);    // 소수점 이하 지리수를 3자리로 설정
cout.setf(ios::fixed);
cout.setf(ios::showpoint);
cout.setf(ios::left);
cout.setf(ios::left | ios::showpoint);
cout.unsetf(ios::uppercase);    // 설정 해제
```

# 텍스트와 이진 파일



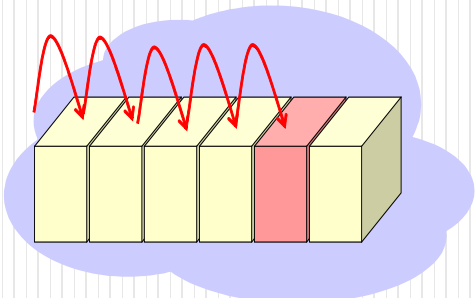
텍스트 파일: 문자로 구성된 파일



이진 파일: 데이터로 구성된 파일

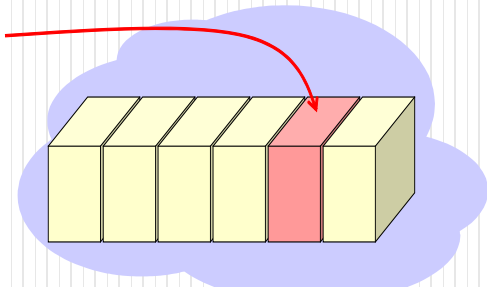
# 순차 파일과 랜덤 파일

- 임의 접근 파일
  - 순차 접근(sequential access)방법
    - 데이터를 파일의 처음부터 순차적으로 읽거나 기록하는 방법



순차접근파일

- 임의 접근(random access)방법
  - 파일의 어느 위치에서든지 읽기와 쓰기가 가능한 방법



임의접근파일



# 파일 쓰기/읽기

- 파일 쓰기

```
#include <iostream>
#include <fstream>
#include <cassert>
using namespace std;
```

```
int main ()
{
```

```
    // Instantiation of an ofstream object
```

```
    ofstream outStrm;
```

```
    // Creation of a file and connecting it to the ofstream object
```

```
    outStrm.open ("file.txt");
```

```
    if (!outStrm.is_open())
```

```
    {
```

```
        cout << "integerFile cannot be opened!";
```

```
        assert (false);
```

```
    }
```

```
    // Writing to the file using overloaded insertion operator
```

10 20 30 40 50 60 70 80 90 100

```
for (int i = 1; i <= 10; i++)
```

```
{
```

```
    outStrm << i * 10 << " ";
```

```
}
```

```
    // Closing the file
```

```
    outStrm.close ();
```

```
    // The ofstream object is destroyed after return statement
```

```
    return 0;
```

```
}
```

# 파일 쓰기/읽기

- 파일 읽기

```
#include <iostream>
#include <fstream>
#include <cassert>
using namespace std;
```

```
int main ()
{
```

```
    int data;
```

```
    // Instantiation of an ifstream object
```

```
    ifstream inStrm;
```

```
    // Connection of the existing file to the ifstream object
```

```
    inStrm.open ("file.txt");
```

```
    if (!inStrm.is_open())
```

```
    {
```

```
        cout << "integerFile cannot be opened!";
```

```
        assert (false);
```

10 20 30 40 50 60 70 80 90 100

```
    }
```

```
    // Reading from the ifstream object and writing to the cout object
```

```
    for (int i = 1; i <= 10; i++)
```

```
    {
```

```
        inStrm >> data ;
```

```
        cout << data << " " ;
```

```
    }
```

```
    // Disconnection of the IntegerFile from the ifstream
```

```
    inStrm.close ();
```

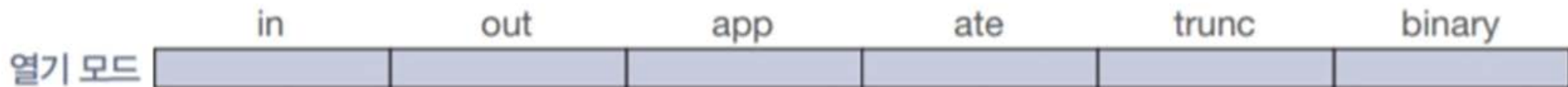
```
    // Destruction of the ifstream object is done after return
```

```
    return 0;
```

```
}
```

# ifstream을 이용한 파일 읽기/쓰기

| 플래그         | 의미                                                 |
|-------------|----------------------------------------------------|
| ios::in     | 입력을 목적으로 열기                                        |
| ios::out    | 출력을 목적으로 열기(원본 내용은 파괴됨)                            |
| ios::app    | 출력을 목적으로 열기, 이때 추가되는 내용은 기존의 파일 뒤에 추가됨(append의 약자) |
| ios::ate    | 파일을 열고 바로 가장 뒤로 이동(at end의 약자)                     |
| ios::trunc  | 파일에 내용이 있으면 파일의 내용을 모두 지우고 열기                      |
| ios::binary | 이진 모드로 파일 열기(기본적으로는 텍스트 모드)                        |



# ifstream을 이용한 파일 읽기/쓰기

```
#include <iostream>
#include <fstream>
#include <cassert>
using namespace std;
```

```
int main ()
{
```

```
    // Variable declaration
```

```
    char ch;
```

```
    // Instantiation of an ifstream object
```

```
    ifstream istrm ;
```

```
    // Opening file1 and testing if it is opened properly
```

```
    istrm.open ("file1", ios :: in);
```

```
    if (!istrm.is_open())
```

```
    {
```

```
        cout << "file1 cannot be opened!" << endl;
```

**Run:**

This is the file that we want to open  
and read its contents.

```
        assert (false);
```

```
    }
```

```
    // Reading file1 character by character and writing to monitor
```

```
    while (istrm.get (ch))
```

```
    {
```

```
        cout.put(ch) ;
```

```
    }
```

```
    // Closing stream
```

```
    istrm.close () ;
```

```
    return 0;
```

```
}
```

# 기존 파일 뒤에 데이터 추가

```
#include <iostream>
#include <fstream>
#include <cassert>
using namespace std;
```

This is the file that we want to open and read its contents.  
October 15, 2015.

```
int main ()
{
    // Instantiate a ostream object
    ofstream ostr;
    // Open file1 and connect it to the ostream object
    ostr.open ("file1", ios :: out | ios :: app);
    if (!ostr.is_open())
    {
        cout << "file1 cannot be opened!";
        assert (false);
    }
}
```

```
// Append the date as a C-string to file1
ostr << "\nOctober 15, 2016.";
// Close the file
ostr.close ();
return 0;
```

```
}
```

# 이진 데이터 쓰기/읽기

```
#include <iostream>
#include <string>
#include <fstream>
#include <cassert>
using namespace std;

int main ()
{
    int int1 = 12325;
    double double1 = 45.78;
    // Creating a new file for output and write the two data types
    ofstream strmOut ("Sample", ios :: out | ios :: binary);
    if (!strmOut.is_open())
    {
        cout << "The file Sample cannot be opened for writing!";
        assert (false);
    }
```

```
    }
    strmOut.write (reinterpret_cast <char*> (&int1), sizeof (int));
    strmOut.write (reinterpret_cast <char*> (&double1), sizeof (double));
    strmOut.close ();

    int int2;
    double double2;
    // Opening the same file for input and reading the two data types
    ifstream strmIn ("Sample", ios :: in | ios :: binary);
    if (!strmIn.is_open())
    {
        cout << "The file Sample cannot be opened for reading!";
        assert (false);
    }
    strmIn.read (reinterpret_cast <char*> (&int2) , sizeof (int));
    strmIn.read (reinterpret_cast <char*> (&double2) , sizeof (double));
    strmIn.close ();
    // Testing the value of stored data types
    cout << "Value of int2: " << int2 << endl;
    cout << "Value of double2: " << double2 << endl;
    return 0;
} // End main
```

Run:

Value of int2: 12325

Value of double2: 45.78